

改良版ソーラーカーパッケージ 処理フロー体験ワークシート

solar_ws0129-main

2026-07-16

使い方

本書は、現在の改良版ソーラーカーパッケージについて、

- PowerShell で何を叩くと、
- どの `launch / script / node` が動き、
- どの `topic / CSV / dashboard / planner` 更新へ流れるか

を、添付の“MPC 処理フロー体験”と同じく、手を動かして追える形でまとめたワークシートである。

PASSO 系列は本書では扱わない。対象はソーラーカー系の各モード、`sim`、`measure`、`live`、`live_wifi`、`forecast`、`identify` である。

- デフォルトは回答表示ありである。
- 穴埋めだけにしたい場合は、プレアンプルの `\answerstrue` を `\answersfalse` に変更する。
- 本書は 2026-07-16 時点のワークツリー、`SolarSim.ps1`、`install` 側の `scripts/solar_control.sh`、`export_rqt_graph.py`、および現在確認できる `core node` 実装を根拠にしている。

Contents

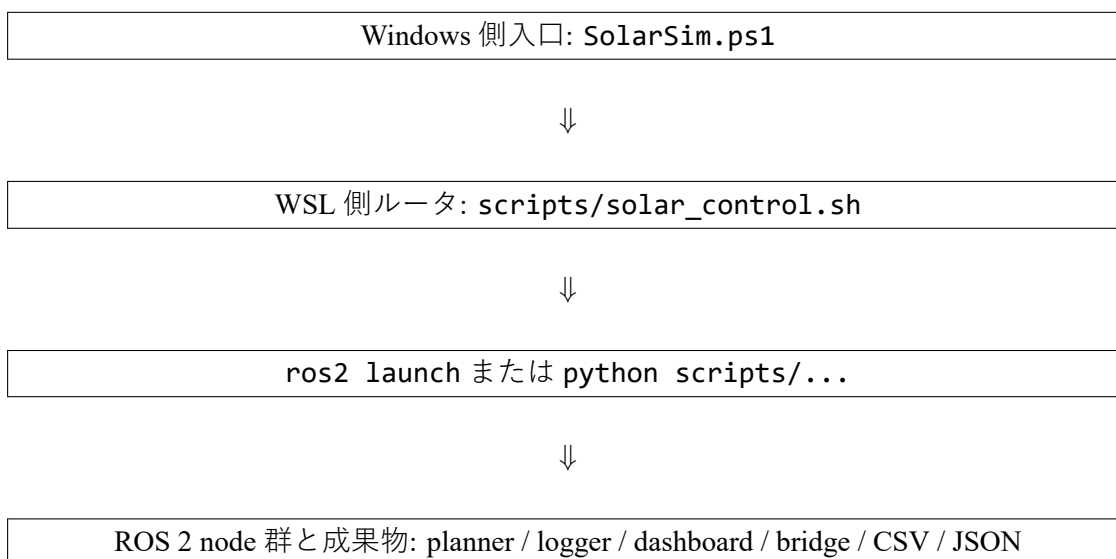
1	まず最初に押さえるべき事実	4
1.1	入口は1つではなく3層ある	4
1.2	入口と実装本体は source 側で対応している	4
1.3	本書の根拠ファイル	4
2	全体フローの骨格	5
2.1	PowerShell から見た action と mode	5
2.2	WSL 側ルータでの mode -> launch 対応	6
2.3	graph exporter が mode ごとに期待する core node	6
2.4	問題 1: 入口の穴埋め	6
3	profile.yaml の読み方	6
3.1	改良版 profile のトップレベル構造	6
3.2	mode ごとに読む profile ブロック	7
3.3	問題 2: profile の対応づけ	7
4	sim モード	8
4.1	sim モードには2種類ある	8
4.2	launch 型 sim の流れ	8
4.3	launch 型 sim で直接確認できる node	8
4.4	offline 型 sim の流れ	8
4.5	offline 型 sim で出る主成果物	9
4.6	sim で重要な topic	9
4.7	問題 3: sim フロー追跡	9
5	measure モード	9
5.1	measure モードの位置づけ	9
5.2	measure の想定 runtime	10
5.3	distance_node がしていること	10
5.4	grade_node がしていること	10
5.5	solar_logger_node が残す列	10
5.6	問題 4: measure の要点	11
6	live モード	11
6.1	live モードの位置づけ	11
6.2	live の想定流れ	11
6.3	mpc_node が live で受ける実測	11
6.4	command bridge に期待される役割	12
6.5	weather / autocal / wind model の役割	12
6.6	問題 5: live の穴埋め	12

7 live_wifi モード	13
7.1 live_wifi は live に WiFi 受送信層を足したもの	13
7.2 profile 上での主役	13
7.3 live_wifi の設計フロー	13
7.4 WiFi bridge の source と責務	13
7.5 live_wifi で追うべき代表 topic / データ	14
7.6 問題 6: live_wifi の要点	14
8 forecast action	14
8.1 forecast は launch ではなく utility 実行	14
8.2 何をしているか	14
8.3 forecast で触る代表パラメータ	14
8.4 問題 7: forecast の穴埋め	14
9 identify action と改良版 vehicle identification	15
9.1 identify action の軽量 pipeline	15
9.2 改良版 vehicle identification の主戦力	15
9.3 template と入力雛形	15
9.4 問題 8: identification	15
10 dashboard と logger の見方	16
10.1 dashboard_node が受けるもの	16
10.2 solar_logger_node が担うこと	16
10.3 問題 9: dashboard / logger	16
11 source ファイルと役割の対応表	16
12 実装監査結果と読むべき姿勢	17
13 総合問題	18
14 最後に	18

1 まず最初に押さえるべき事実

1.1 入口は1つではなく3層ある

現行の操作系は、大きく次の3層に分かれる。



1.2 入口と実装本体は source 側で対応している

2026-07-16 の監査結果: SolarSim.ps1 と scripts/solar_control.sh が使う solar_measurement.launch.py、solar_race_live.launch.py、solar_race_live_wifi.launch.py は、いずれも launch/ に存在する。telemetry / command / wind / state bridge も mpc_solarcar/ に実装され、setup.py の entry point と一致する。本書では、

- PowerShell 入口から launch まで
- 各 node の topic、数式、保存成果物

を同じ source tree に基づいて追跡する。

1.3 本書の根拠ファイル

根拠ファイル	本書での役割
SolarSim.ps1	Windows から叩く最上位入口。action / mode / profile を WSL に渡す。
scripts/solar_control.sh	ROS 起動、sim、forecast、identify、fit、learn、audit を振り分ける本体。
launch/*.launch.py	sim / measure / live / live_wifi の node 構成を定義する直接根拠。
scripts/solar_sim.py	オフライン大会全体 simulation。CSV、detail CSV、upper plan CSV、HTML、JSON を出力。
project_packages/bwsc2027_template/profile.yaml	改良版 profile 構造の代表例。各 mode がどのパラメータ群を使うかの根拠。
mpc_solarcar/mpc_node.py	上位・下位 planner の中心。通常 1 時間の上位再計画と 1 Hz lower command の根拠。
mpc_solarcar/dashboard_node.py	dashboard が何を受け取り、何を API として見せるかの根拠。

根拠ファイル	本書での役割
<code>mpc_solarcar/distance_node.py</code>	<code>/vehicle/speed_kmh</code> から <code>/vehicle/s_km</code> を積分する根拠。
<code>mpc_solarcar/grade_node.py</code>	GPS / altitude / distance から勾配を出す根拠。
<code>mpc_solarcar/solar_logger_node.py</code>	太陽車ログ CSV の列と topic 対応の根拠。
<code>mpc_solarcar/solar_preflight_node.py</code>	solar telemetry / planner freshness から system state / diag / health を作る。
<code>scripts/export_rqt_graph.py</code>	mode 別 core node 群の graph export。
<code>scripts/fetch_weather_forecast.py</code>	forecast action が何をするかの根拠。
<code>scripts/run_identification_pipeline.py</code>	identify action の軽量 pipeline の根拠。
<code>scripts/run_vehicle_identification.py</code>	改良版 generic replay MLE identification の根拠。

2 全体フローの骨格

2.1 PowerShell から見た action と mode

action	mode	到達先
up	sim / measure / live / live_wifi	build -> stop -> start。最終的に <code>ros2 launch mpc_solarcar <launch_file></code> 。
start	同上	既存 build を前提に、その mode の launch だけ起動。
graph	同上	<code>export_rqt_graph.py --mode <mode></code> 。graph exporter が想定 node 群を待って画像化。
simulate	sim を主対象	<code>python scripts/solar_sim.py --profile_yaml ...</code> 。ROS2 launch ではなくオフライン実行。
forecast	主に live / live_wifi 前準備	<code>python fetch_weather_forecast.py --profile_yaml ...</code> 。Open-Meteo 取得。
identify	profile ベース	<code>python run_identification_pipeline.py --profile_yaml ...</code> 。軽量の同定 pipeline。
fit	profile ベース	実ログを用いる高品質 segmented MLE と map shape fit。
learn	profile ベース	CEM で上位目的重みを探索し、検証 profile と TensorBoard log を保存。
audit	全体	inventory、static contract audit、pytest を連続実行。
package / blank	Windows 直接	検証済み solar-only 版 / 同機能の空版を再生成。
log	任意	<code>.run/solar_<mode>.log</code> を tail。

2.2 WSL 側ルータでの mode -> launch 対応

mode	launch file (solar_control.sh が期待する名前)
sim	solarcar_sim.launch.py
measure	solar_measurement.launch.py
live	solar_race_live.launch.py
live_wifi	solar_race_live_wifi.launch.py

2.3 graph exporter が mode ごとに期待する core node

mode	期待 core node
sim	gps_sim_node, mpc_node, solar_state_node, dashboard_node
measure	dashboard_node, solar_logger_node, distance_node, grade_node
live	mpc_node, dashboard_node, solar_logger_node, speed_command_bridge_node
live_wifi	mpc_node, dashboard_node, solar_logger_node, speed_command_bridge_node, telemetry_text_bridge_node, wind_correction_node

2.4 問題 1: 入口の穴埋め

1. Windows 側で最初に叩く入口は **SolarSim.ps1** である。
2. WSL 側で action / mode を解釈する本体は **scripts/solar_control.sh** である。
3. simulate action は **ros2 launch** ではなく、**python scripts/solar_sim.py** を直接呼ぶ。
4. live_wifi の graph exporter が追加で期待する bridge 系 node は **telemetry_text_bridge_node** と **wind_correction_node** である。

3 profile.yaml の読み方

3.1 改良版 profile のトップレベル構造

改良版 profile の代表例 `project_packages/bwsc2027_template/profile.yaml` では、次のブロックが入口になる。

ブロック	主な役割
meta	package 名、目的、注意事項。
paths	route / weather / schedule / maps の実ファイル。多くの script がここを入口にする。
runtime	dashboard host/port、forecast 時間解釈の基準。
logging	共通 log 置き場と rate。
simulation	start 時刻、初期 SoC、出力先、prerace 実行条件。
measurement	distance_node と grade_node の使い方。
identification	同定 input / output ディレクトリ。
live	実運用用。weather / autocal / command bridge / WiFi / wind model を束ねる。
model	車体質量、CdA、Crr、PV、battery 定数。
mpc	上位 / 下位 MPC の地平、重み、制約、再計画周期。

3.2 mode ごとに読む profile ブロック

mode	強く効くブロック
sim	paths、simulation、model、mpc、runtime
measure	measurement、logging、必要なら runtime
live	live.weather、live.autocal、live.command_bridge、model、mpc
live_wifi	live.wifi_bridge、live.wind_model、live.command_bridge、live.weather、model、mpc
forecast	paths.route_waypoints_csv、paths.forecast_csv、live.weather、runtime
identify	identification、paths、model

3.3 問題 2: profile の対応づけ

1. dashboard の host / port は主に **runtime** ブロックで決まる。
2. offline sim の初期 SoC、開始時刻、出力先は主に **simulation** ブロックで決まる。
3. 実運用の command smoothing や accel 制限は **live.command_bridge** に置く。
4. WiFi 受送信先 IP / port は **live.wifi_bridge** に置く。

4 sim モード

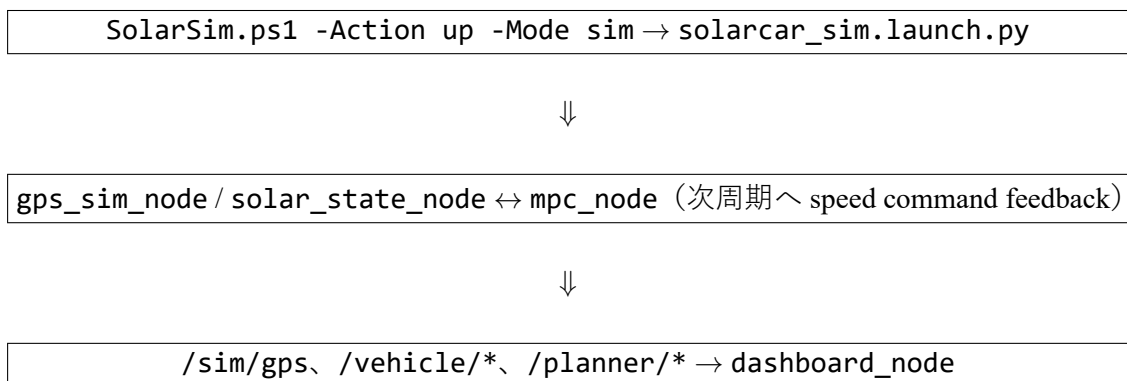
4.1 sim モードには 2 種類ある

1. **launch 型 sim**: SolarSim.ps1 -Mode sim -Action up で起動する ROS2 runtime。
2. **offline 型 sim**: SolarSim.ps1 -Mode sim -Action simulate で実行する scripts/solar_sim.py。

この 2 つは同じ profile を参照するが、役割が違う。

- launch 型 sim は topic を流し、planner と dashboard を起動する。
- offline 型 sim は大会全体 replay / planning 用の CSV、HTML、JSON を出力する。

4.2 launch 型 sim の流れ



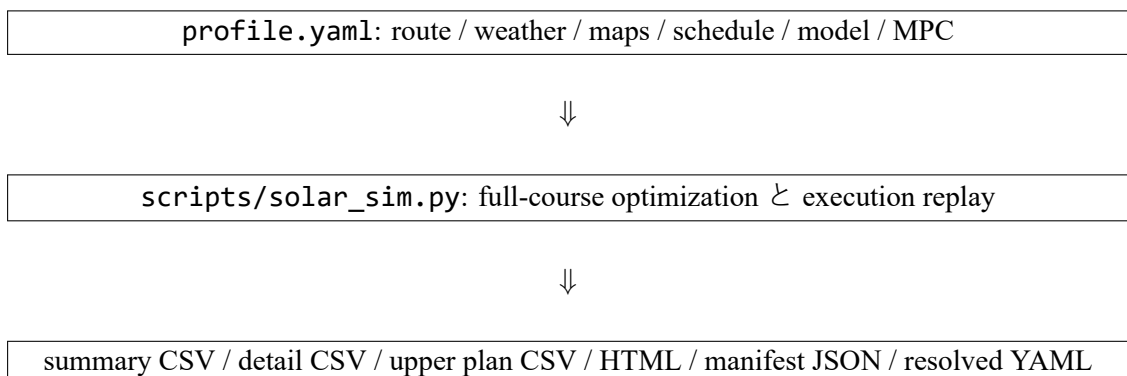
4.3 launch 型 sim で直接確認できる node

`launch/solarcar_sim.launch.py` で確認できるのは次の 3 node である。

1. `gps_sim_node`: `inputs/route_waypoints.csv` と `/planner/speed_cmd` を使って `/sim/gps` を publish する。
2. `mpc_node`: `forecast CSV`、`route profile`、`speed profile`、`drive schedule`、`maps`、`params` を読み、`/planner/speed_cmd`、`/planner/upper_plan`、`/planner/lower_plan`、`/planner/status`などを publish する。
3. `dashboard_node`: `/api/state` を持つ HTTP サーバで、`planner` と `vehicle` の主要状態を可視化する。

4.4 offline 型 sim の流れ

`scripts/solar_sim.py` は ROS2 launch を通らず、profile を読んで大会全体をロールアウトする。



4.5 offline 型 sim で出る主成果物

成果物	内容
out_csv	時刻、距離、speed command、実行速度、SoC、battery 温度、G_poa、外気温、向かい風。
out_detail_csv	電流、電圧、PV 出力、P_pack、各抵抗力、drive / regen 効率など、再現確認用の詳細系列。
out_plan_csv	上位プランナが各再計画時刻で採用した segment 列。
report_html	人間がブラウザで見る概要レポート。
summary_json	finish 到達、final distance、final SoC、CPU 時間などの要約。
resolved_yaml	override を織り込んだ実行時 profile の写し。
latest_simulation_run.json	最新 run の所在を追跡する manifest。

4.6 sim で重要な topic

topic	主 publisher	意味
/planner/speed_cmd	mpc_node	planner が今出した速度指令。
/planner/upper_speed_cmd	mpc_node	上位 planner 出力の代表値。
/planner/upper_plan	mpc_node	上位 plan の配列。dashboard に渡る。
/planner/lower_plan	mpc_node	下位 plan の配列。
/planner/status	mpc_node	状態の数値サマリ。
/planner/env	mpc_node	planner が見ている環境量。
/planner/metrics	mpc_node	電力・制約関連メトリクス。
/sim/gps	gps_sim_node	route 上を進む模擬 GPS。

4.7 問題 3: sim フロー追跡

1. launch 型 sim で /planner/speed_cmd を受けて route 上を進める node は **gps_sim_node** である。
2. offline 型 sim で大会全体の詳細な P_pack、I、V まで出るのは **out_detail_csv** である。
3. 通常 1 時間の上位再計画と 1 Hz lower command の本体は **mpc_node** にある。
4. dashboard の HTTP API endpoint は **/api/state** である。

5 measure モード

5.1 measure モードの位置づけ

measure は、実車や試験系から来る測定を

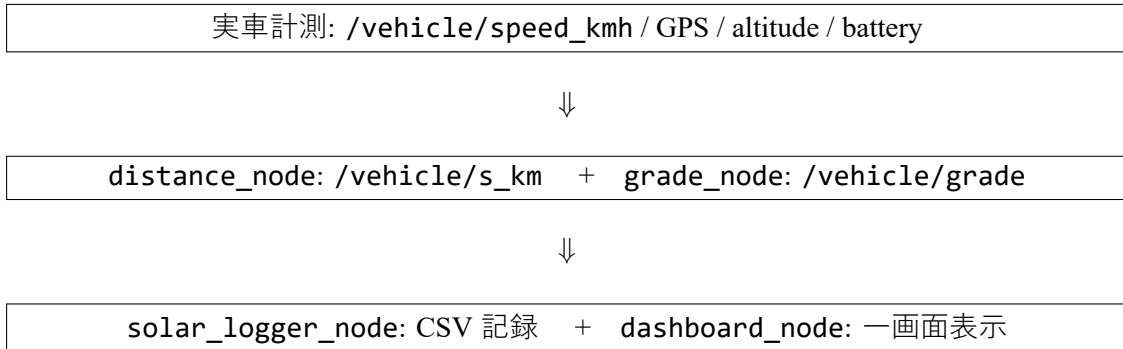
- 距離に積分し、
- 勾配に変換し、
- ログに残し、

- dashboard で見えるようにする

ための mode である。

実装根拠: measurement launch が dashboard_node、solar_logger_node、distance_node、grade_node を profile の measurement ブロックで起動する。本節は推測ではなく、launch と各 node source を直接追跡した処理である。

5.2 measure の想定 runtime



5.3 distance_node がしていること

mpc_solarcar/distance_node.py は、/vehicle/speed_kmh を受け、台形則に近い形で

$$s_{k+1} = s_k + \frac{v_k + v_{k+1}}{2} \frac{\Delta t}{3600}$$

を積分し、/vehicle/s_km を publish する。

したがって measure や live で距離が必要なら、/vehicle/s_km を外から直接入れるか、あるいは distance_node を必ず併走させる。

5.4 grade_node がしていること

mpc_solarcar/grade_node.py は、

- /vehicle/s_km
- /vehicle/speed_kmh
- /vehicle/altitude_m または GPS altitude

を受ける。標高は EMA で平滑化し、距離差が十分たまったときに勾配を更新して /vehicle/grade を出す。

5.5 solar_logger_node が残す列

solar_logger_node は、少なくとも次を記録する。

- ROS 時刻、UTC 壁時計、車速、距離、標高、勾配
- battery SoC / 温度 / 電流 / 電圧、PV 電力
- 実測・補正向かい風、風速・風向・進行方位、planner environment
- 上位 / 下位 speed command、drive mode、model metrics、online 校正值
- WiFi solar / chase 生文字列、system health / state / diag、各 node status

5.6 問題 4: measure の要点

1. /vehicle/s_km を生成する node は **distance_node** である。
2. /vehicle/grade を出す node は **grade_node** である。
3. CSV を残す node は **solar_logger_node** である。
4. measure mode の expected core node は **dashboard_node, solar_logger_node, distance_node, grade_node** である。

6 live モード

6.1 live モードの位置づけ

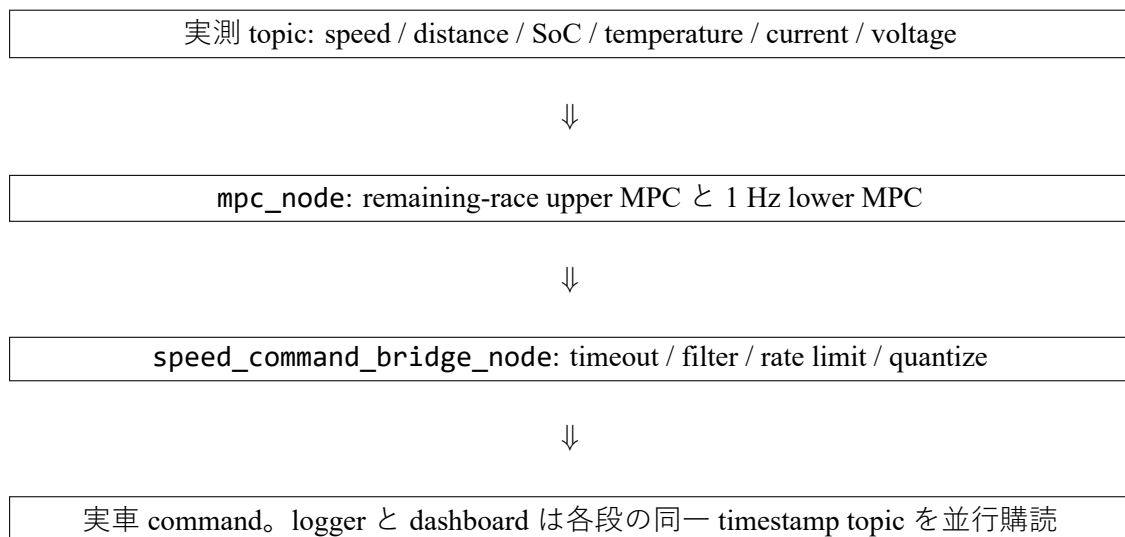
live は、実走中に planner を回し続け、測定を受けて速度指令を出す mode である。改良版 profile では live ブロックの下に、

- weather
- autocal
- command_bridge
- wind_model

がある。

実装根拠: live launch は weather、MPC、自動校正、指令 bridge、logger、dashboard を profile から構成する。mpc_node.py の publish / subscribe と合わせ、実走時の経路を source で確認できる。

6.2 live の想定流れ



6.3 mpc_node が live で受ける実測

mpc_node の solar mode では、少なくとも次の実測を subscribe する。

- /vehicle/s_km
- /vehicle/speed_kmh

- /vehicle/batt_soc
- /vehicle/batt_temp_c
- /vehicle/batt_current_a
- /vehicle/batt_voltage_v

上位 timer は 1 Hz で再計画条件を監視するが、重い上位最適化は `upper_replan_sec`（通常 3600 秒）を満たした時に行う。階層型が有効なら lower timer が `lower_rate_hz`（通常 1 Hz）で指令を更新する。

6.4 command bridge に期待される役割

profile の `live.command_bridge` には、

- `publish_rate_hz`
- `input_timeout_sec`
- `safe_speed_kmh`
- `startup_hold_sec`
- `filter_tau_sec`
- `accel_limit_kmhps`
- `decel_limit_kmhps`
- `speed_deadband_kmh`
- `speed_quantize_step_kmh`
- `max_output_speed_kmh`
- `drive_mode_min_hold_sec`

があり、実運用時に“出力が暴れない”ようにする層である。

6.5 weather / autocal / wind model の役割

- `live.weather`: 実走中の位置または `fallback` 緯度経度を使い、一定周期で予報を取得する想定。
- `live.autocal`: 走行中 / 停止中 / 昼夜条件に応じて、`solar gain`、`drive power gain`、補機電力推定をじわっと更新する想定。
- `live.wind_model`: 風予報の分散成長、相関距離、`planning quantile` を使い、補正 forecast を `planner` に供給する想定。

6.6 問題 5: live の穴埋め

1. `live` で `mpc_node` が通常 1 Hz で指令更新するのは **下位 planner** である。
2. 実測の SoC は `/vehicle/batt_soc` topic で入る。
3. 速度指令の平滑化や `accel` 制限は主に `live.command_bridge` が担う。
4. `live` モードの `graph exporter` 期待 core node には `speed_command_bridge_node` が含まれる。

7 live_wifi モード

7.1 live_wifi は live に WiFi 受送信層を足したもの

live_wifi では、live の planner / logger / dashboard に加え、

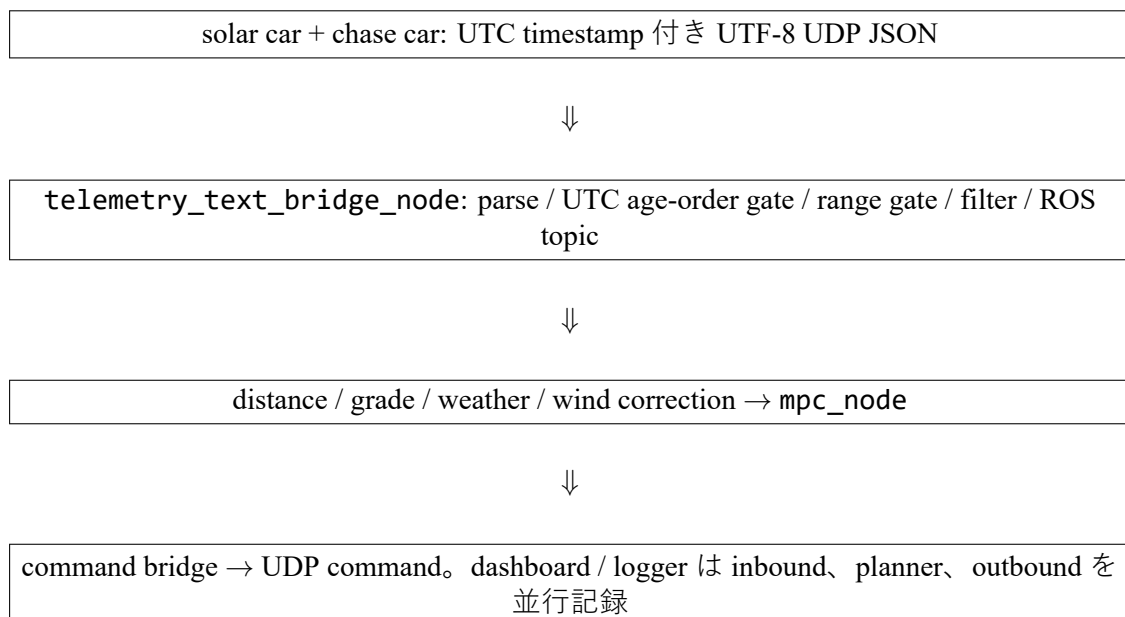
- text telemetry bridge
- outbound command bridge
- wind correction

が追加される想定である。

7.2 profile 上での主役

ブロック	役割
live.wifi_bridge	受信 bind host/port、送信先 host/port、送受信有効化、speed / distance / battery / wind のフィルタ時定数。
live.command_bridge	planner 出力を実車へ渡す直前の平滑化・安全化。
live.weather	chase 側 GPS などから予報取得位置を決める。
live.wind_model	corrected forecast CSV を作る想定。

7.3 live_wifi の設計フロー



7.4 WiFi bridge の source と責務

実装済み: telemetry_text_bridge_node.py、speed_command_bridge_node.py、wind_correction_node.py、solar_state_node.py は mpc_solarcar/ に存在する。launch/solar_race_live_wifi.launch.py と setup.py の entry point も一致する。WiFi UTF-8 文字列受信から検証・filter、ROS topic、planner、指令平滑化、UDP 送信まで source で追跡できる。送信側 Raspberry Pi は NTP / chrony で UTC 同期し、毎 packet に timestamp_utc または ts_unix を付ける。既定では 5 秒超の遅延、2 秒超の未来時刻、重複・順序逆転を topic 更新前に拒否する。

7.5 live_wifi で追うべき代表 topic / データ

- inbound 側: speed、distance、battery、風、chase GPS
- 時刻監査: /telemetry/solar_source_ts_unix、/telemetry/chase_source_ts_unix、受信 UTC
- planner 内部: /planner/speed_cmd、/planner/upper_plan、/planner/status
- outbound 側: output_speed_topic、output_drive_mode_topic または UDP / 文字列
- artifact 側: live_forecast_raw.csv、live_forecast_corrected.csv、runtime logs

7.6 問題 6: live_wifi の要点

1. WiFi の bind host / port を置くのは **live.wifi_bridge** である。
2. corrected forecast CSV の出力先を持つのは **live.wind_model** である。
3. live_wifi の graph exporter 期待 core node は live に加えて、次の 2 node を持つ。
telemetry_text_bridge_node と **wind_correction_node**

8 forecast action

8.1 forecast は launch ではなく utility 実行

SolarSim.ps1 -Action forecast は、最終的に scripts/fetch_weather_forecast.py --profile _yaml ... を呼ぶ。

8.2 何をしているか

1. profile を開く。
2. live.weather と runtime から、forecast days、step minutes、timezone、tcell gain を決める。
3. route waypoint CSV があれば先頭点の緯度経度を使う。
4. 無ければ fallback 緯度経度を使う。
5. Open-Meteo forecast を取得する。
6. paths.forecast_csv へ保存する。

8.3 forecast で触る代表パラメータ

- forecast_days
- step_minutes
- timezone_name
- fallback_latitude / fallback_longitude
- tcell_gain

8.4 問題 7: forecast の穴埋め

1. forecast action は **ros2 launch** ではなく **fetch_weather_forecast.py** を呼ぶ。
2. route waypoint CSV が存在しないときに使う緯度経度は **fallback 緯度経度** である。

9 identify action と改良版 vehicle identification

9.1 identify action の軽量 pipeline

SolarSim.ps1 -Action identify は、install 側の run_identification_pipeline.py を呼ぶ。これは入力ディレクトリ内のファイル有無に応じて、次を順に回す軽量 pipeline である。

1. gps_track.csv があれば route / profile 同定
2. battery_rest.csv があれば OCV curve 構築
3. battery_pulse.csv があれば Rint map 構築
4. panel_sweep.csv があれば panel / MPPT map 構築
5. drive_timeseries.csv があれば vehicle parameter fit

9.2 改良版 vehicle identification の主戦力

現在の改良版では、generic replay MLE を扱う scripts/run_vehicle_identification.py の方が重要である。これは次を入口にする。

- package 内 data/identification/identification_manifest.yaml
- normalized_replay_log_csv
- 既存または理論根拠のある maps/*.csv

9.3 template と入力雛形

template	用途
templates/identification_manifest_template.yaml	generic replay MLE の入口 manifest。
templates/identification/observed_replay_log_template.csv	時刻、距離、速度、気象、battery 観測を揃えた replay 入力。
templates/identification/battery_pulse_template.csv	Rint 系の pulse 測定。
templates/identification/battery_rest_template.csv	OCV 休止測定。
templates/identification/drive_timeseries_template.csv	駆動系時系列。
templates/identification/gps_track_template.csv	route / profile 作成用 GPS。
templates/identification/panel_sweep_template.csv	panel / MPPT map 作成用掃引。
templates/identification/source_map_catalog_template.csv	既知 map の根拠整理。

9.4 問題 8: identification

1. 軽量 pipeline の入口 script は run_identification_pipeline.py である。
2. 改良版 generic replay MLE の入口 script は run_vehicle_identification.py である。
3. replay ベースで最初に揃えるべき雛形 CSV は observed_replay_log_template.csv である。

10 dashboard と logger の見方

10.1 dashboard_node が受けるもの

dashboard_node は少なくとも次を subscribe する。

- planner: speed command、upper speed command、throttle command、drive mode、status、env、metrics、upper plan、lower plan
- vehicle: speed、SoC、battery 温度、battery 電流、battery 電圧、distance
- system: state、diag、MPC state、health
- location: /sim/gps
- trajectory: /planner/trajectory

つまり dashboard は、“planner が何を考えているか”と“実測がどうなっているか”の両方を見せる。

10.2 solar_logger_node が担うこと

logger は、「後から再現・同定・異常解析できること」が役割である。したがって、dashboard に見えてい
るだけではなく、

- speed command
- throttle command
- measured speed / distance
- battery channels
- system state / diag / health

を CSV として固定化する。

10.3 問題 9: dashboard / logger

1. dashboard の HTTP API は **/api/state** である。
2. planner の配列 plan を dashboard に渡す topic は **/planner/upper_plan** と **/planner/lower_plan** である。
3. 後解析用 CSV の主担当は **solar_logger_node** である。

11 source ファイルと役割の対応表

ファイル	読むと何が分かるか
SolarSim.ps1	ユーザ操作の入口、WSL distro 解決、引数の受け渡し。
scripts/solar_control.sh	action と mode の最終分岐。
launch/solarcar _sim.launch.py	現物として確認できる sim launch 構成。
scripts/solar_sim.py	大会全体 simulation の処理順、出力ファイル、実行モデル。

ファイル	読むと何が分かるか
mpc_solarcar/mpc_node.py	planner の入出力 topic、上位 replan / 1 Hz lower、hierarchical 構成。
mpc_solarcar/model.py	物理モデル。
mpc_solarcar/dashboard_node.py	どの値が dashboard に乗るか。
mpc_solarcar/distance_node.py	speed -> distance 積分。
mpc_solarcar/grade_node.py	GPS / altitude -> grade。
mpc_solarcar/solar_logger_node.py	太陽車ログ CSV の列定義。
mpc_solarcar/solar_preflight_node.py	solar telemetry と planner の鮮度から system state を作る。
scripts/fetch_weather_forecast.py	予報取得のロジック。
scripts/run_identification_pipeline.py	軽量同定 pipeline。
scripts/run_vehicle_identification.py	replay MLE 同定と report 生成。

12 実装監査結果と読むべき姿勢

重要な結論: 現在の package は、planner、offline sim、profile、logger / dashboard / distance / grade に加え、次の runtime source もワークツリー直下に揃っている。

- solar_measurement.launch.py
- solar_race_live.launch.py
- solar_race_live_wifi.launch.py
- speed_command_bridge_node.py
- telemetry_text_bridge_node.py
- wind_correction_node.py
- solar_state_node.py
- solar_profile.py, weather_utils.py

今チームが使うときは

1. PowerShell 入口と profile 構造を理解する
2. launch と node source で topic / parameter を追う
3. source inventory の hash と配布 manifest で版を照合する

という 3 段で読むのが正しい。未確認の install 側バイナリを前提にする必要はない。

13 総合問題

問題 10: mode 横断追跡

次の空欄を埋めよ。

1. 大会全体の detail CSV と upper plan CSV を出す主役 script は **scripts/solar_sim.py** である。
2. 実測車速から距離へ変換する node は **distance_node** である。
3. 実走中の上位再計画と 1 Hz lower command の中核 node は **mpc_node** である。
4. WiFi 受送信を束ねる設定ブロックは **live.wifi_bridge** である。
5. Open-Meteo 予報取得 utility は **fetch_weather_forecast.py** である。
6. 改良版 replay MLE 同定は **run_vehicle_identification.py** を読む。

問題 11: フローを文章で説明せよ

次の 2 つを、自分の言葉で説明できれば、本 package の入口理解はかなり進んでいる。

1. `SolarSim.ps1 -Action up -Mode sim` を叩いた直後から、dashboard に値が出るまでの流れ。
2. `SolarSim.ps1 -Action simulate` を叩いた直後から、summary CSV / detail CSV / upper plan CSV / HTML が保存されるまでの流れ。

14 最後に

本 package を運用する上で重要なのは、“MPC の数式” だけではない。実際には、

- どの mode を叩いたか
- その mode がどの profile ブロックを読むか
- topic がどこから入り、どこへ出るか
- 最後にどの CSV / dashboard / command へ着地するか

を一続きで追えることが、運用の強さになる。

本書はその全体地図である。数値を代入して全サイクルを解く問題・解答・解答用紙は、docs/complete_flow_workbook/ に分冊して収録する。